# ES6 Classes



`class` (ES2015) gives JavaScript a familiar, declarative way to build objects and inheritance chains — but it is **syntactic sugar over the prototype system**, not a new object model. Interviewers love classes precisely because the sugar hides mechanics they want you to explain: where methods actually live, what `super` does, why `this` breaks when a method is detached, and what private fields really guarantee. If you can map every class feature back to prototypes, you can answer almost any class question.

## Classes are sugar over prototypes

A class and the classic constructor-function pattern produce the same runtime shape. Methods declared in the class body are installed on the constructor's `.prototype`, and instances delegate to it — exactly what you would wire up by hand.

JS  Copy

```
class User {
  constructor(name) { this.name = name; }
  greet() { return `Hi, ${this.name}`; }
}

// Equivalent pre-ES6 form:
function UserFn(name) { this.name = name; }
UserFn.prototype.greet = function () { return `Hi, ${this.name}`; };

const a = new User('Ada');
console.log(a.greet());                                  // Hi, Ada
console.log(typeof User);                                // function
```

The key facts an interviewer wants: `typeof User === 'function'` (a class **is** a function object), and `greet` lives once on `User.prototype`, **shared** by every instance — not copied per object. That is why `a.hasOwnProperty('greet')` is `false`: the lookup walks the prototype chain.

## constructor, fields, and methods

The `constructor` runs on `new` and initializes the instance. **Instance fields** (class fields) are assigned per-instance, as if written at the top of the constructor, so they become own enumerable properties. Regular methods go on the prototype.

JS Copy

```
class Point {
  x = 0;                // instance field (own property)
  y = 0;
  constructor(x, y) { this.x = x; this.y = y; }
  dist() { return Math.hypot(this.x, this.y); }
}
const p = new Point(3, 4);
console.log(p.dist());              // 5
console.log(Object.keys(p));        // ['x', 'y'] — fields are own; dist is
```

## extends and super

`extends` sets up the prototype chain between the two constructors and their prototypes. In a derived constructor you **must call** `super( ... )` **before touching** `this` — `super()` invokes the parent constructor, which is what actually allocates and initializes `this` in a subclass. `super.method()` calls the parent's version of a method (useful when overriding).

JS Copy

```
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);                // parent constructor first
    this.breed = breed;
  }
  speak() { return `${super.speak()} (a bark)`; }  // extend, don't replac
}
const d = new Dog('Rex', 'Lab');
console.log(d.speak());                          // Rex makes a sound (a bark
console.log(d instanceof Dog, d instanceof Animal); // true true
```

## static members

`static` methods and fields live on the **constructor itself**, not on instances — use them for factories, constants, and utilities that operate on the type rather than one object. Inside a static method `this` refers to the class.

JS Copy

```
class Circle {
  static PI = 3.14159;
  static from(r) { return new Circle(r); }   // named factory
  constructor(r) { this.r = r; }
  get area() { return Circle.PI * this.r ** 2; }
}
const c = Circle.from(2);
console.log(c.area.toFixed(2));  // 12.57
console.log(Circle.PI);          // 3.14159 — on the class, not on `c`
```

## Private fields (#x)

A field prefixed with `#` is **hard private**: it is accessible only inside the class body, is not enumerable, does not appear in `Object.keys`, and referencing `obj.#x` outside the

class is a **syntax error** — not merely a convention like the old `_name`. This lets you enforce invariants that outside code cannot bypass.

JS Copy

```
class Account {
  #balance = 0;                          // real privacy, engine-enforced
  deposit(n) { this.#balance += n; return this; }
  get balance() { return this.#balance; }
  static isAccount(obj) {
    return #balance in obj;              // brand check: never throws
  }
}
const acc = new Account();
acc.deposit(100).deposit(50);
console.log(acc.balance);              // 150
console.log(Object.keys(acc));         // [] — #balance is invisible
console.log(Account.isAccount(acc));   // true
console.log(Account.isAccount({}));    // false
```

Why it matters: before `#`, "private" was an underscore naming convention or a closure trick. `#` moves the guarantee into the engine — even `Object.getOwnPropertyNames` won't reveal it. The `#x in obj` form is a safe **brand check** that answers "was this object made by my class?" without throwing on foreign objects.

## Getters and setters

`get` / `set` define **accessor properties** on the prototype, so a computed value or a validated write looks like a plain property to callers. A setter is the natural place to guard a private field.

JS Copy

```
class Temp {
  #c = 0;
  get fahrenheit() { return this.#c * 9 / 5 + 32; }
  set celsius(v) {
    if (typeof v !== 'number') throw new TypeError('number required');
    this.#c = v;
  }
```

```
t.celsius = 100;
console.log(t.fahrenheit);   // 212
```

## new.target, TDZ, and strict mode

Three smaller facts interviewers use as tie-breakers:

- `new.target` is the constructor invoked with `new` (or `undefined` for a plain call). It lets a function detect misuse or an abstract base class refuse direct instantiation.
- **Classes are not hoisted** the way function declarations are — the binding exists but sits in the **Temporal Dead Zone** until the declaration executes, so using it earlier throws a `ReferenceError`.
- **Class bodies always run in strict mode**, regardless of surrounding code. That is why an unbound extracted method sees `this === undefined` instead of the global object.

JS Copy

```
function Widget() {
  if (new.target === undefined) throw new Error('call with new');
}
new Widget();                                    // ok
try { Widget(); } catch (e) { console.log(e.message); } // call with new

try { new Foo(); } catch (e) { console.log(e.constructor.name); } // Refer
class Foo {}
```

## Classes vs factory functions / closures

Both encapsulate state; they trade off differently. **Classes** give a shared prototype (memory-efficient methods), real `instanceof`, `extends`, and `#`-privacy — but every method is `this`-dependent, so detaching a method loses its receiver. **Factory functions with closures** capture private state in a scope, need no `this`, and return

methods that are safe to destructure and pass around — but each object gets its own copy of every method (more memory) and there is no prototype chain or `instanceof`.

JS Copy

```
function createCounter(start = 0) {
  let count = start;                  // private via closure, no `this`
  return {
    inc() { return ++count; },
    get value() { return count; },
  };
}
const { inc } = createCounter(10);
console.log(inc());    // 11 — safe even when detached
```

Rule of thumb: reach for **classes** when you have many instances, a real inheritance hierarchy, or want `instanceof`; reach for a **factory/closure** when you want the simplest airtight privacy, no `this` foot-guns, and objects that survive being torn apart (React handlers, event callbacks).

## Common interview questions

**Q: Are classes hoisted?** The *binding* is hoisted (the name is reserved in scope) but it stays in the TDZ, so you cannot reference the class before its declaration — unlike a `function` declaration, which is fully hoisted and callable earlier. Effectively: treat classes as not usable before their line.

**Q: What is `super`?** In a constructor, `super( ... )` calls the parent constructor and is what initializes `this` in a subclass — you must call it before using `this`. In a method, `super.foo()` calls the parent's `foo` via the prototype chain, letting you extend rather than replace behavior.

**Q: How are methods stored?** Regular and `get`/`set` methods live once on the class's `.prototype`, shared by all instances (so `instance.hasOwnProperty(method)` is `false`). `static` methods live on the constructor. **Instance fields** and **arrow-function fields** are per-instance own properties, created fresh for every object.

**Q: Is a class a function?** Yes — `typeof MyClass === 'function'`. `class` is sugar; the constructor is a function object whose `.prototype` holds the methods.

class body. The old `_name` underscore is only a convention.

## Gotchas / traps

**Forgetting `super()` in a derived constructor.** Any use of `this` — or returning — before `super()` throws, because in a subclass `this` doesn't exist until the parent constructor runs.

JS Copy

```
class Base { constructor() { this.ok = true; } }
class Bad extends Base {
  constructor() { this.x = 1; super(); }   // this before super()
}
try { new Bad(); }
catch (e) { console.log(e.constructor.name); } // ReferenceError
```

**Arrow methods vs prototype methods.** A method written as an arrow **class field** (`inc = () => {}`) is an own per-instance property that closes over `this`, so it survives detaching — but it is *not* on the prototype, costs memory per instance, and cannot be overridden via `super`. A normal method is shared on the prototype but loses `this` when detached. Choose deliberately.

JS Copy

```
class Counter {
  count = 0;
  inc() { this.count++; }              // prototype, this-dependent
  incArrow = () => { this.count++; };  // own field, this-bound
}
const c = new Counter();
console.log(Counter.prototype.hasOwnProperty('inc'));      // true
console.log(Counter.prototype.hasOwnProperty('incArrow')); // false
console.log(c.hasOwnProperty('incArrow'));                 // true
```

This is the classic bug behind `onClick={obj.handle}` in React. Fix by binding in the constructor, using an arrow field, or wrapping in an inline arrow.

JS Copy

```
const c = new Counter();
const loose = c.inc;
try { loose(); } catch (e) { console.log(e.constructor.name); } // TypeErr

const bound = c.incArrow;    // arrow field keeps `this`
bound();
console.log(c.count);        // 1 — works detached
```

**Calling a class without `new`.** Unlike an old constructor function (which would silently run with `this` bound wrong), a `class` **throws** `TypeError: Class constructor cannot be invoked without 'new'`. That is a safety feature, and a common trick-question output.

## QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  constructor(name) { super(name); }
  speak() { return `${super.speak()} (a bark)`; }
}
const d = new Dog('Rex');
console.log(d.speak());
```

- undefined makes a sound (a bark)
- Rex makes a sound (a bark)
- TypeError: super.speak is not a function

Check answer

2. What does this log?

JAVASCRIPT Copy

```
class Account {
  #balance = 0;
  deposit(n) { this.#balance += n; return this; }
  get balance() { return this.#balance; }
  static isAccount(obj) { return #balance in obj; }
}
const acc = new Account();
acc.deposit(100).deposit(50);
console.log(acc.balance, Object.keys(acc).length, Account.isAccount({}), Acc
```

- 150 1 false true
- 150 0 false true
- 150 0 true false
- 50 0 false true

Check answer

3. Which statement about ES6 classes is correct?

A class declaration is fully hoisted like a function declaration, so it is callable before its line.

- class introduces a new object model separate from prototypes, and each instance gets its own copy of every method.
- typeof MyClass is 'object', because a class is not a function.
- A class is a function object whose methods live once on .prototype (shared by all instances), and its binding is hoisted but sits in the Temporal Dead Zone, so it can't be used before its declaration runs.

Check answer

Previous
The Prototype Chain & Inheritance

Next
The Event Loop

## LEARN MORE

### ARTICLES & DOCS

Classes (MDN)
MDN

### VIDEOS




JavaScript ES6 classes (OOP)
YouTube




Learn classes in 6 minutes
YouTube




ES6 classes — constructors, statics…
YouTube

Hand-picked references for this lesson.

JavaScript Interview Kit